

CSE 1113

Computer Programming

Sumaya Kazary

Assistant Professor

Department of Computer Science and Engineering
Dhaka University of Engineering & Technology, Gazipur

Acknowledgement

Thanks to the authors of all the books and online tutorials used in this slide.

C structures and unions

Structure - keyword **struct**

- A structure is a convenient way of **grouping several pieces of related information** together.
- **A structure is a collection of variables under a single name.**
- Provide a way of storing many different values in variables of potentially **different types** **under the same name**.
- Each has a name which is used to select it from the structure.
- A structure can be defined as a new named type, thus extending the number of available types.
- **It can use other structures, arrays or pointers as some of its members**, though this can get complicated unless you are careful.

Defining a Structure

- The format for defining a structure is

```
struct Tag {  
    Member1;  
    Member2;  
    .  
    .  
    Member N;  
};
```

the name of the entire type of structure

the variables within the structure.

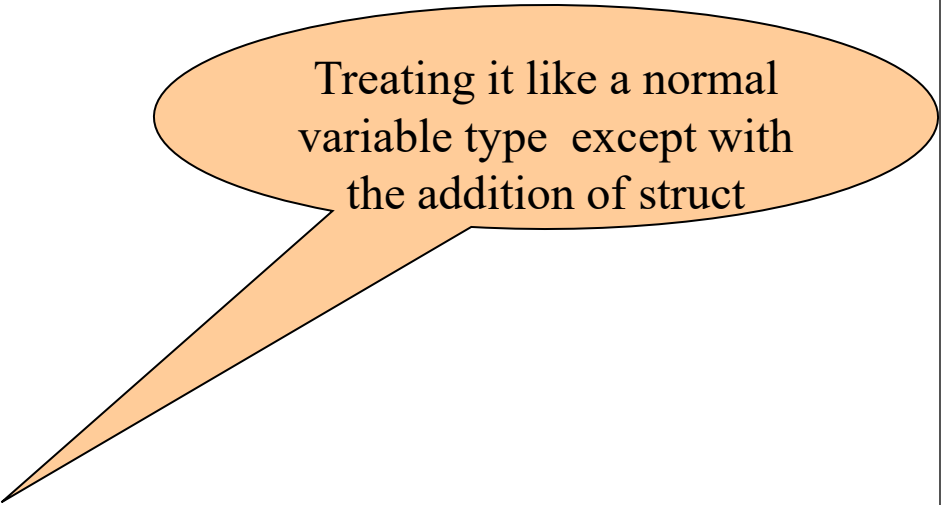
- To actually create a single structure, the syntax is

```
struct Tag name_of_single_structure;
```

Example

```
struct student {  
    int id;  
    int dept;  
    float cgpa;  
};
```

- struct student std1;



Treating it like a normal variable type except with the addition of struct

Declaring Structures (struct)

Does Not Reserve Space

```
struct my_example  
{  
    int label;  
    char letter;  
    char name[20];  
};
```

/* The name "my_example" is
called a structure tag */

Reserves Space

```
struct my_example  
{  
    int label;  
    char letter;  
    char name[20];  
} mystruct ;
```

Accessing Members of a Structure

- Each member of a structure can be used just like a normal variable,(but its name will be a bit longer)
- To return to the examples on last slide, member **id** of structure **student** will behave just like a normal int, however we refer to it by the name **student.id**
 - Here the dot(.) is an operator which selects a member from a structure.

Input a student's marks of 5 courses and display the total.

```
void main(){
    int i;
    float sum=0.0;
    struct student pp;
    printf("Enter the Std. Id#:" );
    scanf("%d",&pp.id);
    for(i=0;i<5;i++){
        printf("Course#%d marks ?", i+1);
        scanf("%f", &pp.marks[i]);
        sum+= pp.marks[i];
    }
    printf("The total is=%f", sum);
}
```

```
struct student{
    int id;
    float marks[5];
};
```

Example: program to add two distances in Ft-inch format.

```
void main(){
    float sum_inch;
    int sum_feet,ft;
    struct distance d1,d2;
    printf("Enter the 1st distance D1(ft-inch):");
        scanf("%d%f",&d1.feet,&d1.inch);
    printf("Enter the 2nd distance D2 (ft-inch):");
        scanf("%d%f",&d2.feet,&d2.inch);
    sum_inch= d1.inch+d2.inch;
    if(sum_inch>=12)
        { ft=1;
          sum_inch-=12; }

    sum_feet=d1.feet+d2.feet+ft;
    printf("D1+D2=%d feet %f in.", sum_feet, sum_inch);
}
```

```
struct distance{
    int feet;
    float inch;
};
```

Structure Within Structure

```
void main( ){  
    room dining;  
    float sum_l ,sum_w, area,A;
```

```
    struct distance{  
        int feet;  
        float inch;  
    };  
    struct room{  
        distance length;  
        distance width;  
    };
```

```
    printf("Enter the room's length(ft-inch):");  
    scanf("%d %f",&dining.length.feet, &dining.length.inch);  
    printf("Enter the room's width(ft-inch):");  
    scanf("%d %f",&dining.width.feet,&dining.width.inch);
```

```
    sum_l= ((dining.length.feet *12)+ dining.length.inch);  
    sum_w= ((dining.width.feet *12)+ dining.width.inch);
```

```
    area=sum_l*sum_w;
```

```
    A=area/144.0;
```

```
    printf("\n The area of the room is %f Sq.ft",A);
```

```
    getch();
```

```
}
```

Uses of Structures

1. Structures are generally useful whenever a lot of **data needs to be grouped together**
2. It is also a good way of **representing certain types of information**. Complex numbers in mathematics inhabit a two dimensional plane (stretching in real and imaginary directions).

These could easily be represented here by:-

```
typedef struct {  
    double real;  
    double imag;  
} Complex;           /* The "alias" is Complex */  
  
Complex C_no;        /* Create a struct variable */
```

3. Apart from holding data, **structures can be used as members of other structures**. Arrays of structures are possible, and are a good way of storing lists of data with regular fields, such as databases.
4. Another possibility is a **structure whose fields include pointers to its own type**. These can be used to build chains (programmers call these linked lists), trees or other connected structures. These are rather daunting to the new programmer, so we won't deal with them here.

Unions

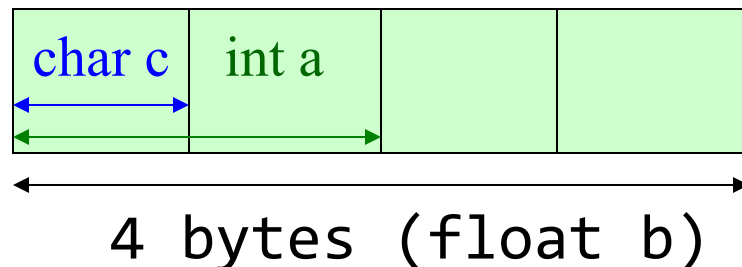
- Unions are like structures except that all the variables share the same memory.
- When a union is declared the compiler allocates enough memory for the largest data-type in the union.
- It's like a giant storage chest where you can store one large item, or a small item, but never the both at the same time.
- Thus, although a **union** may contain many members of different types, it can handle only one member at a time.
- The '.' operator is used to access different variables inside a union also.

Memory Layout

```
struct rect{  
    int a;  
    float b;  
    char c;  
};
```

It takes 7 bytes in the storage.

```
union rect{  
    char c  
    int a;  
    float b; };
```



It takes 4 bytes in the storage.

Example

```
struct candidate{  
    char name[20];  
    int mar_status;  
    union gardian{  
        char f_name[30];  
        char h_name[25];  
    };  
};
```

Questions?